

METHODOLOGY REPORT

非工程背景的 AI 協作開發方法論

用 AI 從零交付四套系統的實戰總結

Nexus • Matrix • CRM • YYTW

架構先行

80 / 20 人力配置

驗收即紀律

紅隊自我審查

作者:馮若陽 Marcus Feng • 資深營運經理

協作工具:Claude (規劃 / 文件) + Claude Code (執行)

用途:記錄一套「非工程背景也能用 AI 交付系統」的可複製方法

為什麼寫這份報告

這份報告想拆解一件事：**「會用 AI」和「能用 AI 交付一套上線系統」是兩回事**。差別不在用哪個工具,而在三件事——架構先行、80/20 的人力配置、把驗收當成核心紀律。

過去一年,我以非工程背景,用 AI 協作開發陸續交付了四套系統:顧問工作台 Nexus、營運數據平台 Matrix、全端 CRM,以及個人 side project「YYTW」社群平台。其中 CRM 通過了公司資訊部的資安檢測(Security Review E→A)。

我看過太多人跟 AI 來回溝通數十次還無法聚焦,做出來的東西東拼西湊、最後不了了之。這份報告的目的,是把我這四個專案裡反覆驗證有效的做法整理成一套**可複製的方法**——讓任何「知道營運痛點、但不會寫程式」的人,也能用 AI 把系統做出來,而且做得可信、可維護、可交接。

一句話總結

“**如果你連跟 AI 都有溝通障礙,**
要怎麼跟真人合作?”

— 這是我前東家主管的一句話,我一直記在心上。
AI 協作的本質,是「把需求說清楚」的能力;說不清楚,換誰來做都做不好。

這份報告分成兩部分:

先用一張表,看四個專案的進化軌跡;
再用三個核心發現,拆解這套方法為什麼成立、又該怎麼複製。

四個專案,一條進化軌跡

這四個專案不是各自獨立的成果,而是同一套方法逐步成熟的證據:

專案	性質	耗時	在方法演進中的角色
Nexus 顧問資訊整合中心	公司內部	第一版約 2 週	第一次嘗試,摸索「架構先行」的價值
Matrix 營運數據分析中心	公司內部	第一版約 1 週	複用 Nexus 經驗,速度加倍
CRM 全端營運平台	公司內部	共約 2 個月 (搭建 2 週)	最複雜:含 SA 架構、顧問訪談、流程梳理;加入真人工程師驗收,做到資安 A 級
YYTW 社群市集平台	個人專案	約 2+2 週	驗證這套方法能遷移到全新情境(社群電商)

從 2 週到 1 週(變快)、到 2 個月最複雜專案(加入企業級驗收)、再到遷移個人專案—— 速度與複雜度的成長曲線,本身就說明這套方法在成熟。

接下來三頁,拆解支撐這條軌跡的**三個核心發現**:架構先行、驗收即紀律、給非工程人的可複製習慣。

核心發現一:架構先行,第一個 Prompt 最重要

最重要的工作發生在「動手之前」。把整個產品的 full picture 想清楚、寫成完整架構,是整個專案成敗的關鍵。連公司資訊部的工程師都向我證實:架構最重要。

很多人跟 AI 溝通失敗,根因是**自己也不清楚產品的全貌長什麼樣子**。於是跟 AI 來回數十次、每次補一點,結果方向飄移、功能東拼西湊。

我的做法相反:在寫任何功能之前,先把所有需要的功能、可能的使用情境、權限怎麼設、功能之間如何搭配,全部想清楚,讓 AI(Claude)產出一份完整的 **架構 .md 文件**。這份文件成為整個專案的「施工藍圖」——之後每個功能都對照它檢查,確保開發不失焦。

關鍵心法:把架構交給 AI 拆解,而不是自己一步步指揮

有了完整架構之後,我不會一個指令一個指令地命令 AI。我讓 AI **自己依照架構去拆解步驟、按不同的 Phase 分階段製作**。實務上,即使中間偶有偏離,最後通常都會回到正軌——因為有那份架構當錨點。

實務對比

無架構的人:跟 AI 來回數十次還在原地打轉,因為不知道終點在哪,AI 再強也只是亂槍打鳥。

架構先行的人:第一個 Prompt 就包含完整架構,AI 自行分階段執行,方向穩定。

差別不在 AI,在「人有沒有先把 full picture 想清楚」。

這套架構先行,在四個專案的規模

正因為架構想得夠完整,這四套系統的功能密度都不低:

系統	主要功能數	代表性模組
Nexus	約 11 項	總覽儀表板、院校資料庫、歷屆榜單查詢、選校推薦產生器、AI 設定、權限管理、使用量監測…
Matrix	約 9 項	總覽 Dashboard、即時績效監控、個別顧問分析、營運風險預警、產品組成統計、來源績效統計…
YYTW	約 12 項	商品主頁、上架商品、個人衣架、訊息聊天、討論板、品項收藏、註冊審核管理、後台管理…
CRM	完整生命週期	學生 360 詳情頁(7 分頁)、申請看板(9 狀態)、字數帳本、文件版控、多角色權限…

核心發現二:驗收,是整套方法的核心

AI 會「以為自己成功了」。所以我的工作重點不是「叫 AI 做」,而是「**確認 AI 做的是對的**」。這套驗收紀律,是我作為非工程背景的人,能交付出資安 A 級系統的真正原因。

分階段驗收:每個節點都讓 AI 先自我挖錯

配合「架構先行 → 分 Phase 製作」,每完成一個階段就是一個驗收節點。在每個節點,我會先請 AI **自己去挖自己寫的 bug**,例如檢查:

- 某個功能是否真的能順利執行?
- 串接 Supabase 資料庫,資料有沒有真的寫入成功?
- 重設密碼的驗證信,有沒有真的寄出?
- 剛完成平台轉移後,既有帳戶密碼需重設——那**已綁定 MFA 雙重驗證的管理員**,會不會反而被擋在外面?

建立 Agent 隊伍 + 紅隊,模擬攻擊自己

除了逐項自查,我還會請 AI **建立 Agent 隊伍**(模擬假管理員、假社員)把整個流程跑一遍,並**規劃紅隊**做 smoke test、**攻擊自己的系統**找漏洞,再自我修補。最後,一定有真人實際驗收。

真實案例:驗證救了我

這是「自我挖錯」真正發揮作用的一次。我口頭提醒 Claude Code 去檢查,它查了之後發現 **重設密碼的驗證信確實沒寄成功**;更關鍵的是,它**自己額外發現了一筆「MFA 管理員可能會被擋」的風險**並向我報告。我隨即請它設計 Agent 團隊(假管理員、假社員),從註冊、登入、後台管理…全部重跑一遍,列出所有問題,再和我一起逐一解決。
如果只相信「程式碼看起來對」,這些問題會直接帶到上線——只有實際驗證抓得到。

我認為最理想的工作配置:80% AI + 20% 人,缺一不可

我的結論是:**有需求的單位,要有「能與 AI 協作開發」的人力**,再交由真人工程師擔任復盤角色、給現場人員指導,產品才能落地得夠快。

80% 由 AI 執行,20% 的人力絕對不可省。那 20% 是判斷力、是責任、是「在不可逆的關口踩煞車」——這正是 AI 還無法取代的部分。CRM 專案就是這個配置的成果:我主導 + AI 執行 + 資訊部工程師復盤驗收。

核心發現三:給「想用 AI 建系統」的非工程人

這套方法可複製,關鍵不是技術,是四個習慣:知道自己要什麼、把需求說清楚、主動挖坑給自己跳、永遠站在使用者角度。

1. 先知道「自己需要什麼」與「手上有什麼工具」

這是起點。沒有清楚的需求,工具再多也沒用。先盤點:我要解決什麼營運問題?我手上有什麼工具可以組合?

2. 溝通,是 AI 協作的核心技能

在最早開始就要盡可能規劃出所有功能、所有使用情境(例如: **CRM 名單如果重複了怎麼辦?**),並請 AI 預留擴充性——明確告訴它「這邊我之後可能要做什麼功能」,讓架構保留彈性。

3. 主動挖坑給自己跳(壓力測試)

我會刻意去測試系統的脆弱點。下面這個案例,是我主動測「資料會不會被覆寫」時抓到的真實問題:

真實案例:墨爾本大學的「分身」

在嘗試把歷屆榜單與各大專院校排名合併時,我發現一個資料治理問題:部分學生資料因為顧問先前輸入的校名沒有統一,系統吃不到對應,就直接新建了一所學校——例如「The University of Melbourne」和「University of Melbourne」明明是同一所,卻被系統當成兩所不同的學校。

解法(營運懂資料、AI 執行偵測、人定規則):我請 AI 重新設計偵測機制,列出校名重複度、系所名稱重複度、國家重複度、排名重複度都高的疑似重複項,再由人工確認後合併。後來我進一步設立「驗證優先度機制」,確保資料不會被無聲覆寫——至少在覆寫前必須有系統提示。

這正是「非工程背景但懂營運」的價值:我知道資料哪裡會出錯,AI 負責把偵測規則實作出來。

4. 永遠站在使用者角度想 UI / UX

設計時我會不斷自問:如果我是使用者,我會怎麼操作?我會不會找不到某個功能?從 A 到 C 之間,有沒有機會再縮短路徑?減少操作摩擦,和減少錯誤一樣重要。

這四個專案證明了一件事:一個不寫程式的人,確實能主導交付達業界標準的系統——前提是把 AI 當成「需要被驗收的高速產出者」,而不是「可以全盤信任的黑盒子」。
真正的價值,來自人保留的三樣東西:判斷力、責任、與懷疑的紀律。